

Neon Ark Creature Management System

Capstone Project Documentation

COSC 4301 - Modern Programming

Nicolas Canales

Spring 2026

GitHub Repository:

<https://github.com/ncanales12/NeonArkProject4>

Table of Contents

1. Project Overview
2. Technologies Used
3. System Architecture
4. Database Setup
5. Running the Application
6. CLI Features
7. REST API Endpoints
8. Validation and Error Handling
9. Soft Delete Implementation
10. API Testing
11. Screenshots and Testing Evidence
12. Troubleshooting
13. Newcomer Tips
14. Conclusion

1. Project Overview

The Neon Ark Capstone project is a Java Spring Boot REST API and command-line application designed to manage creatures, feeding schedules, observations, habitats, and administrative users inside the Neon Ark facility. The project demonstrates backend API development, PostgreSQL database integration, Docker containerization, DTO usage, validation handling, and a professional menu-driven CLI interface.

The application allows users to register creatures, rename creatures, perform soft deletes, manage observations, search feeding schedules by time, and view administrative system users. The project was developed using Java 17, Spring Boot, PostgreSQL, Docker, Gradle, and IntelliJ IDEA.

The goal of the project was to create a clean and organized backend application that follows modern software development practices while providing a user-friendly command-line experience.

```
=====
      NEON ARK CLI SYSTEM
=====

1. List all creatures
2. View creature by ID
3. Register new creature
4. Rename creature
5. Remove creature
6. View creature observations/notes
7. Find creatures by feeding time
```

Figure 1. Main CLI menu interface (Part 1)

```
3. Register new creature
4. Rename creature
5. Remove creature
6. View creature observations/notes
7. Find creatures by feeding time

--- Admin Only ---
8. View all system users

0. Exit
-----
Select an option:
```

Figure 2. Main CLI menu interface (Part 2)

2. Technologies Used

The Neon Ark Capstone project was developed using the following technologies and tools:

Backend Technologies

- Java 17
- Spring Boot
- Spring Data JPA
- Hibernate

Database Technologies

- PostgreSQL 16
- Docker

Development Tools

- IntelliJ IDEA
- Gradle
- PowerShell
- GitHub

The project uses Spring Boot for REST API development, Hibernate/JPA for database interaction, PostgreSQL as the relational database, and Docker for database containerization.

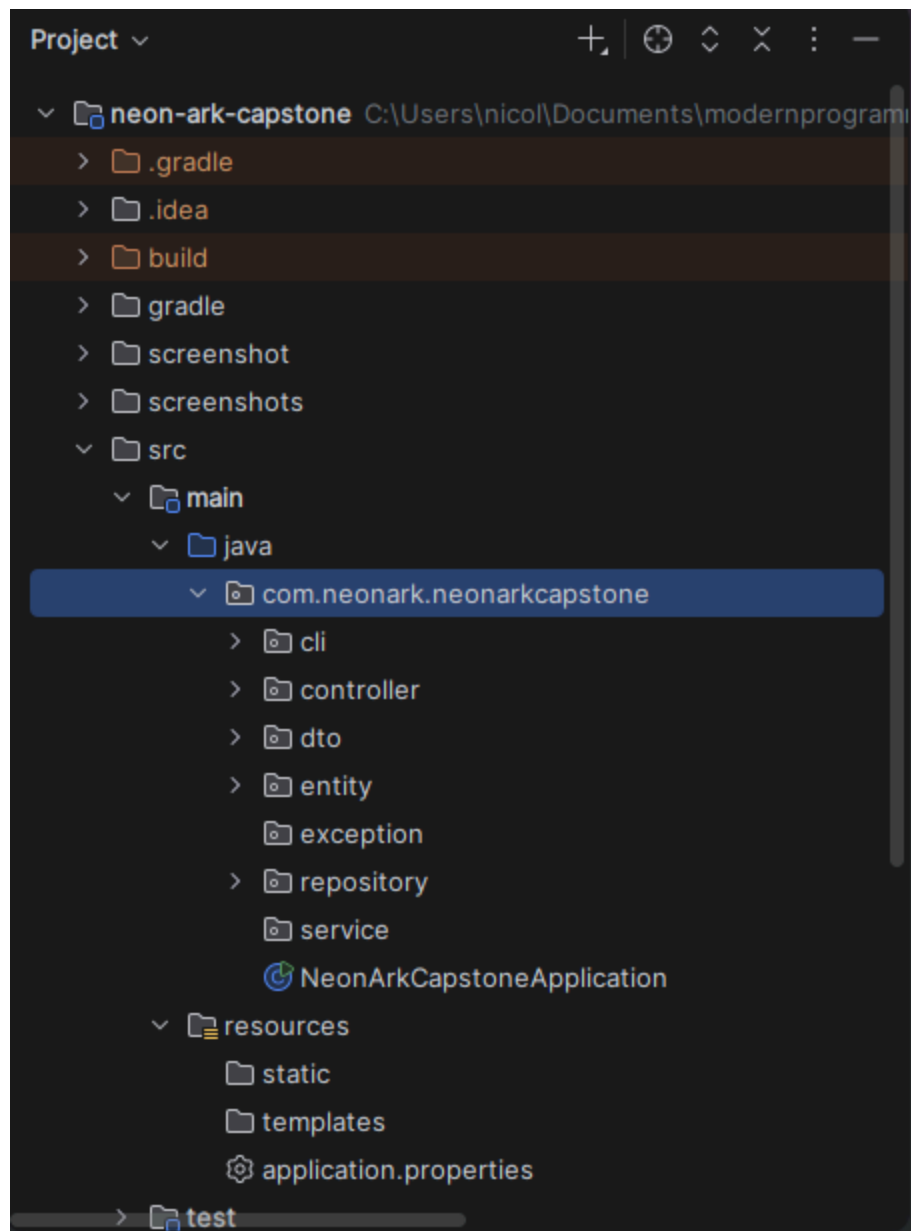


Figure 3. IntelliJ project structure

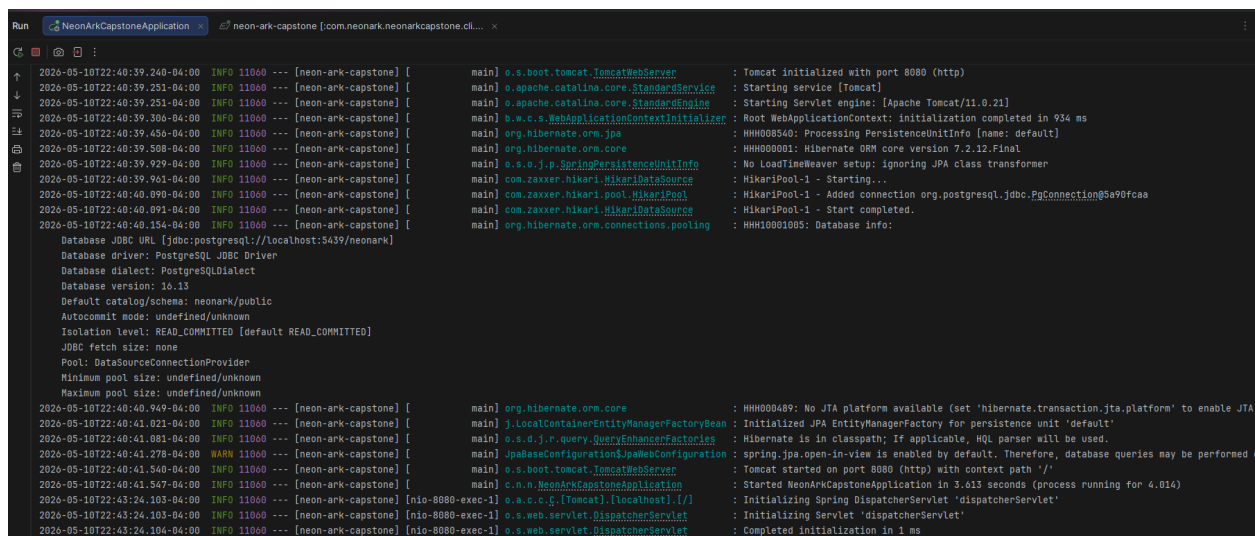
3. System Architecture

The Neon Ark application follows a layered backend architecture using controllers, repositories, DTOs, entities, and a CLI client.

The controller layer manages REST API endpoints and HTTP request handling. The repository layer handles database access using Spring Data JPA. DTO classes are used to control API responses and avoid exposing internal entity structures directly. Entity classes map Java objects to PostgreSQL database tables using Hibernate and JPA annotations.

The command-line interface communicates with the REST API using Java `URLConnection` requests. The CLI sends GET, POST, PUT, and DELETE requests to the Spring Boot backend and displays formatted tables to the user.

The application also includes validation handling, confirmation prompts, and soft delete functionality for creature records.



```
Run NeonArkCapstoneApplication neon-ark-capstone [com.neonark.neonarkcapstone.cli... x
2020-05-10T22:40:39.240-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2020-05-10T22:40:39.251-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.apache.catalina.core.AnnounceService : Starting service [Tomcat]
2020-05-10T22:40:39.251-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.apache.catalina.core.AnnounceService : Starting Service engine: [Apache Tomcat/11.0.21]
2020-05-10T22:40:39.300-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.s.c.s.WebApplicationInitializer : Root WebApplicationContext: initialization completed in 934 ms
2020-05-10T22:40:39.450-04:00 INFO 11060 --- [neon-ark-capstone] [main] org.hibernate.orm.jpa : HHH000540: Processing PersistenceUnitInfo [name: default]
2020-05-10T22:40:39.508-04:00 INFO 11060 --- [neon-ark-capstone] [main] org.hibernate.orm.core : HHH000001: Hibernate ORM core version 7.2.12.Final
2020-05-10T22:40:39.929-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.s.s.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup; ignoring JPA class transformer
2020-05-10T22:40:39.961-04:00 INFO 11060 --- [neon-ark-capstone] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2020-05-10T22:40:40.090-04:00 INFO 11060 --- [neon-ark-capstone] [main] com.zaxxer.hikari.HikariPool : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@5a90fca5
2020-05-10T22:40:40.091-04:00 INFO 11060 --- [neon-ark-capstone] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2020-05-10T22:40:40.154-04:00 INFO 11060 --- [neon-ark-capstone] [main] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
Database JDBC URL [jdbc:postgresql://localhost:5439/neonark]
Database driver: PostgreSQL JDBC Driver
Database dialect: PostgreSQLDialect
Database version: 16.13
Default catalog/schema: neonark/public
Autocommit mode: undefined/unknown
Isolation level: READ_COMMITTED [default READ_COMMITTED]
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2020-05-10T22:40:40.840-04:00 INFO 11060 --- [neon-ark-capstone] [main] org.hibernate.orm.core : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA)
2020-05-10T22:40:41.021-04:00 INFO 11060 --- [neon-ark-capstone] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2020-05-10T22:40:41.081-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.s.s.j.r.query.NucleusEntityManagerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
2020-05-10T22:40:41.270-04:00 WARN 11060 --- [neon-ark-capstone] [main] jpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed
2020-05-10T22:40:41.540-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2020-05-10T22:40:41.547-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.s.s.NeonArkCapstoneApplication : Started NeonArkCapstoneApplication in 3.613 seconds (process running for 4.014)
2020-05-10T22:43:24.103-04:00 INFO 11060 --- [neon-ark-capstone] [nio-8080-exec-1] o.s.c.c.g.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2020-05-10T22:43:24.103-04:00 INFO 11060 --- [neon-ark-capstone] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2020-05-10T22:43:24.104-04:00 INFO 11060 --- [neon-ark-capstone] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
```

Figure 4. Spring Boot application startup

4. Database Setup

The Neon Ark application uses PostgreSQL 16 as the backend relational database. PostgreSQL was containerized using Docker to simplify setup and development.

The database container runs locally and connects to the Spring Boot application through JDBC configuration inside the application.properties file.

The database stores creatures, habitats, observations, feeding schedules, and system users.

Hibernate and Spring Data JPA automatically manage schema updates using the `ddl-auto=update` configuration.

```
spring.datasource.url=jdbc:postgresql://localhost:5439/neonark
spring.datasource.username=postgres
spring.datasource.password=postgres
```

The Docker container was started using the following command:

```
docker run --name neon_ark_postgres `
-e POSTGRES_PASSWORD=postgres `
-e POSTGRES_DB=neonark `
-p 5439:5432 `
-d postgres:16
```

```
PS C:\Users\nicol\Documents\modernprogramming\COSC_4301_Spring_2026_Canales\neon-ark-capstone> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
498211a8fd83   postgres:16   "docker-entrypoint.s..." 3 weeks ago   Up 21 hours   0.0.0.0:5439->5432/tcp, [::]:5439->5432/tcp   neon_ark_postgres
b1ea11147b94   postgres:16   "docker-entrypoint.s..." 6 weeks ago   Up 3 hours   0.0.0.0:5436->5432/tcp, [::]:5436->5432/tcp   monster_db
```

Figure 5. PostgreSQL Docker container running

5. Running the Application

The Neon Ark application requires both the PostgreSQL Docker container and the Spring Boot backend application to be running.

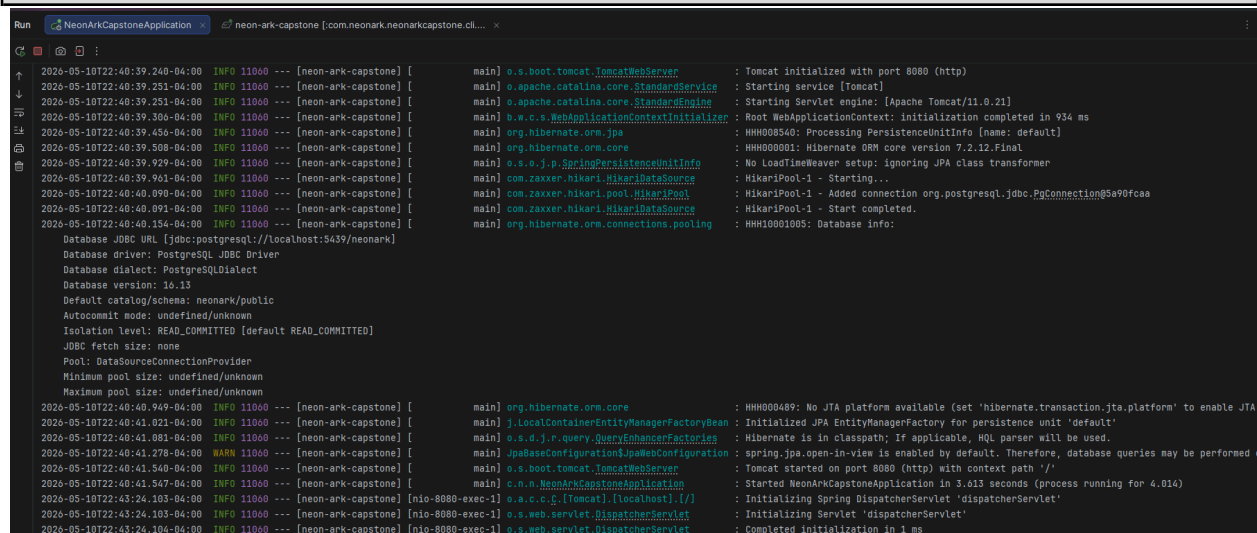
First, the PostgreSQL Docker container must be started. After the database is running, the Spring Boot application can be launched from IntelliJ IDEA by running the NeonArkCapstoneApplication.java file.

Once the backend is running successfully, the command-line interface can be launched by running Main.java.

The application starts on port 8080 and communicates with the PostgreSQL database on port 5439.

NeonArkCapstoneApplication.java

Main.java



```
Run NeonArkCapstoneApplication x neon-ark-capstone [com.neonark.neonarkcapstone.cl... x
2026-05-10T22:40:39.240-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-05-10T22:40:39.251-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-05-10T22:40:39.251-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.21]
2026-05-10T22:40:39.306-04:00 INFO 11060 --- [neon-ark-capstone] [main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 934 ms
2026-05-10T22:40:39.450-04:00 INFO 11060 --- [neon-ark-capstone] [main] org.hibernate.orm.jpa : HHH000540: Processing PersistenceUnitInfo [name: default]
2026-05-10T22:40:39.508-04:00 INFO 11060 --- [neon-ark-capstone] [main] org.hibernate.orm.core : HHH000001: Hibernate ORM core version 7.2.12.Final
2026-05-10T22:40:39.929-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.s.e.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-05-10T22:40:39.961-04:00 INFO 11060 --- [neon-ark-capstone] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2026-05-10T22:40:40.090-04:00 INFO 11060 --- [neon-ark-capstone] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnection@5a90fcaa
2026-05-10T22:40:40.154-04:00 INFO 11060 --- [neon-ark-capstone] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2026-05-10T22:40:40.154-04:00 INFO 11060 --- [neon-ark-capstone] [main] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
Database driver: PostgreSQL JDBC Driver
Database dialect: PostgreSQLDialect
Database version: 16.13
Default catalog/schema: neonark/public
Autocommit mode: undefined/unknown
Isolation level: READ_COMMITTED [default READ_COMMITTED]
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-05-10T22:40:40.949-04:00 INFO 11060 --- [neon-ark-capstone] [main] org.hibernate.orm.core : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA)
2026-05-10T22:40:41.021-04:00 INFO 11060 --- [neon-ark-capstone] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-05-10T22:40:41.081-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.s.d.j.r.query.QueryEnhancerFactories : Hibernate is in classpath; If applicable, HQL parser will be used.
2026-05-10T22:40:41.278-04:00 WARN 11060 --- [neon-ark-capstone] [main] jpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed
2026-05-10T22:40:41.540-04:00 INFO 11060 --- [neon-ark-capstone] [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-05-10T22:40:41.547-04:00 INFO 11060 --- [neon-ark-capstone] [main] c.n.n.NeonArkCapstoneApplication : Started NeonArkCapstoneApplication in 3.613 seconds (process running for 4.014)
2026-05-10T22:43:24.103-04:00 INFO 11060 --- [neon-ark-capstone] [nio-8080-exec-1] o.s.c.e.t.Tomcat.[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-05-10T22:43:24.103-04:00 INFO 11060 --- [neon-ark-capstone] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-05-10T22:43:24.104-04:00 INFO 11060 --- [neon-ark-capstone] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
```

Figure 6. Spring Boot backend successfully running

6. CLI Features

The Neon Ark CLI provides a menu-driven interface that communicates directly with the Spring Boot REST API.

The CLI supports:

- Listing all creatures
- Viewing creatures by ID
- Registering creatures
- Renaming creatures
- Soft deleting creatures
- Viewing observations
- Searching feeding schedules
- Viewing administrative users

The CLI also includes:

- Input validation
- Confirmation prompts
- Error handling
- Professional table formatting
- User-friendly status messages

The command-line interface uses Java `HttpURLConnection` to send HTTP requests to the backend API and display the returned JSON data in formatted tables.

```

=== All Creatures ===

```

ID	Name	Habitat	Status
2	Phoenix	None	ACTIVE
3	Unicorn	Crystal Forest	ACTIVE
7	TestCreature	None	ACTIVE
5	Dragon	None	REMOVED
4	Unicorn	Crystal Forest	REMOVED
9	Shadow Beast	None	ACTIVE
8	Shadow Beast Prime	None	REMOVED

```

=====

```

Figure 7. Listing all creatures in the CLI

```

-----
Select an option: 4
Enter creature ID to rename: 10
Enter new creature name: Nebula Wolf Prime
Are you sure you want to rename this creature? (Y/N): Y

```

```

=== Creature Renamed ===

```

ID	Old Name	New Name	Habitat	Status
10	Nebula Wolf	Nebula Wolf Prime	None	ACTIVE

Figure 8. Creature rename functionality

```

-----
Select an option: 5
Enter creature ID to remove: 10
Are you sure you want to remove this creature? (Y/N): Y

```

```

=== Creature Removed ===

```

ID	Name	Habitat	Status
10	Nebula Wolf Prime	None	REMOVED

Figure 9. Soft delete functionality using REMOVED status

7. REST API Endpoints

The Spring Boot backend exposes REST API endpoints that support the CLI workflow. Each CLI menu option maps to one HTTP request.

Creature Endpoints

GET /api/creatures

GET /api/creatures/{id}

POST /api/creatures

PUT /api/creatures/{id}/name

DELETE /api/creatures/{id}

Observation Endpoints

GET /api/creatures/{id}/observations

POST /api/creatures/{id}/observations

Feeding Endpoint

GET /api/feedings?time={HH:MM}

Admin Endpoint

GET /api/admin/users

POST /api/admin/users

```
PS C:\Users\nicol\Documents\modernprogramming\COSC_4301_Spring_2026_Canales\neon-ark-capstone> Invoke-RestMethod -Uri "http://localhost:8080/api/creatures" -Method GET
```

id	name	habitatName	status
2	Phoenix		ACTIVE
3	Unicorn	Crystal Forest	ACTIVE
7	TestCreature		ACTIVE
5	Dragon		REMOVED
4	Unicorn	Crystal Forest	REMOVED
9	Shadow Beast		ACTIVE
8	Shadow Beast Prime		REMOVED
10	Nebula Wolf Prime		REMOVED

Figure 10. PowerShell API test for creature endpoint

8. Validation and Error Handling

The Neon Ark application includes validation and error handling to improve reliability and user experience.

The backend validates:

- Invalid creature IDs
- Duplicate creature names
- Invalid feeding schedule times
- Blank request values
- Invalid requests

The CLI also supports:

- Confirmation prompts
- Cancelled operations
- Friendly error messages

HTTP response codes are used throughout the application:

- 200 OK
- 201 Created
- 400 Bad Request
- 404 Not Found
- 409 Conflict

This validation system prevents invalid data from being stored in the database and improves overall application stability.

```
--- Admin Only ---  
8. View all system users  
  
0. Exit  
-----  
Select an option: 3  
Enter creature name: Nebula Wolf  
Conflict. That creature name already exists.
```

Figure 11. Duplicate creature validation handling

```
-----  
Select an option: 7  
Enter feeding time (HH:MM): cat  
Bad request. Check your input.
```

Figure 12. Invalid feeding input validation

9. Soft Delete Implementation

The Neon Ark application uses a soft delete strategy for creature removal.

Instead of permanently deleting creature records from the database, the application changes the creature status to REMOVED. This allows records to remain available for tracking, auditing, and historical purposes.

Soft delete behavior helps preserve important data while still indicating that a creature has been removed from active operations.

The CLI and REST API both display ACTIVE and REMOVED statuses to the user.

```
-----
Select an option: 5
Enter creature ID to remove: 10
Are you sure you want to remove this creature? (Y/N): Y

=== Creature Removed ===
ID      Name                Habitat      Status
-----
10      Nebula Wolf Prime       None         REMOVED
```

Figure 13. Creature status changed to REMOVED during soft delete

10. API Testing

PowerShell Invoke-RestMethod commands were used to test all backend REST API endpoints.

Testing included:

- Creature retrieval
- Creature registration
- Creature renaming
- Soft delete functionality
- Feeding schedule searches
- Observation retrieval
- Administrative user retrieval
- Validation and error handling

The API tests confirmed that the backend endpoints responded correctly and returned proper HTTP status codes and JSON responses.

```
PS C:\Users\nicol\Documents\modernprogramming\COSC_4301_Spring_2026_Canales\neon-ark-capstone> Invoke-RestMethod -Uri "http://localhost:8080/api/feedings?time=08:00" -Method GET

Id           : 1
FeedingTime  : 08:00:00
FoodType     : Moon berries
CreatureId   : 3
CreatureName : Unicorn
```

Figure 14. Feeding schedule API testing

```
PS C:\Users\nicol\Documents\modernprogramming\COSC_4301_Spring_2026_Canales\neon-ark-capstone> Invoke-RestMethod -Uri "http://localhost:8080/api/admin/users" -Method GET

fullName : Mara Vale
email    : mara.vale@neonark.test
phone    : 555-0101
roleName : ADMIN
id       : 1

fullName : Jonas Reed
email    : jonas.reed@neonark.test
phone    : 555-0102
roleName : STAFF
id       : 2
```

Figure 15. Administrative user API testing

11. Screenshots and Testing Evidence

The Neon Ark Capstone project includes screenshots demonstrating successful functionality across the CLI application, REST API backend, Docker database container, and IntelliJ project structure.

The screenshots provide evidence that:

- The application runs successfully
- API endpoints function correctly
- Validation handling works properly
- The PostgreSQL Docker container is active
- The CLI interface behaves as expected
- Administrative users display correctly
- Feeding schedules and observations work properly

```
-----
Select an option: 6
Enter creature ID: 3

=== Creature Observations ===
ID      Creature ID  Author      Created At      Note
-----
1       3           Nicolas     2026-05-07T21:27:28.392649  Creature appears calm today.
```

Figure 16. Observation retrieval in the CLI

```
Select an option: 7
Enter feeding time (HH:MM): 08:00

=== Feedings At 08:00 ===
ID      Time      Food      Creature ID  Creature Name
-----
1       08:00:00  Moon berries  3           Unicorn
```

Figure 17. Feeding schedule retrieval

```
Select an option: 8

=== System Users ===
ID      Full Name      Email                      Phone      Role
-----
1       Mara Vale      mara.vale@neonark.test    555-0101   ADMIN
2       Jonas Reed    jonas.reed@neonark.test   555-0102   STAFF
```

Figure 18. Administrative user table in the CLI

12. Troubleshooting

Several common issues were encountered and resolved during development.

Database Connection Issues

At various points during development, incorrect PostgreSQL ports and Docker container configurations caused connection failures. This was resolved by verifying the Docker container status and ensuring the application.properties file matched the correct PostgreSQL port.

Spring Boot Startup Errors

Incorrect entity mappings and DTO mismatches occasionally caused Spring Boot startup failures. These were resolved by updating entity fields, repository methods, and controller responses.

Validation Problems

Invalid feeding schedule times and duplicate creature names initially caused inconsistent behavior. Validation handling was added to return proper HTTP response codes and user-friendly CLI messages.

CLI Formatting Issues

The command-line interface originally displayed raw JSON responses. Professional table formatting was later added to improve readability and usability.

13. Newcomer Tips

Several recommendations can help future developers understand and work with the Neon Ark project more easily.

First, always ensure the PostgreSQL Docker container is running before starting the Spring Boot application. If the database is not active, the backend will fail to connect.

Second, verify that the `application.properties` file contains the correct database port and credentials.

Third, test backend endpoints with PowerShell or Postman before troubleshooting CLI issues. Since the CLI communicates with the REST API, backend problems can often appear as CLI problems.

Finally, keeping DTOs separate from entity classes helps simplify API responses and keeps the application easier to maintain and expand.

14. Conclusion

The Neon Ark Capstone project successfully demonstrates backend software development using Java Spring Boot, PostgreSQL, Docker, REST APIs, DTOs, validation handling, and CLI integration.

The application supports creature management, feeding schedules, observations, administrative users, soft delete functionality, validation handling, and formatted command-line interaction.

Throughout development, the project evolved from a basic API into a more polished and professional backend application with improved architecture, validation, testing, and usability.

The project successfully meets the requirements of the Modern Programming Capstone assignment while demonstrating practical backend software engineering concepts and development workflows.